Solid End-to-End Functional Test Flow: User Account & Product Catalog

Flow ID: `E2E-AC-CAT-FLOW-01`

Flow Name: Unified Shopper Journey: Onboarding, Account Setup, Catalog Exploration, Variant Validation & Recommendation Personalization

Modules Covered: Module 1 (User Accounts & Profile Management) & Module 2 (Product Catalog, Search, Filtering & Recommendations)

Primary Objective: Validate the complete, continuous functional lifecycle of a new shopper—from account registration, profile setup, and session authentication through real-time catalog discovery, faceted filtering, PDP variant matrix selection, and dynamic AI-powered recommendation profiling.

Severity / Priority: P1 / Blocker

Target Execution Environment: Staging / Pre-Production

Estimated Run Time: Manual: ~15 mins | Automated (Playwright): ~45 seconds

1. Flow Overview & Architecture Diagram

```
[ Step 1: Onboarding ]
  User Registration -> Secure Password Enforcement -> Email Verification Token
         |
         ▼
[ Step 2: Session & Security ]
  JWT Issue & Cookie Storage -> Auto/Manual Login -> Session Persistence
         |
         ▼
[ Step 3: Profile & Address Setup ]
  Update Name/Phone -> Save Default Shipping Address -> Verify XSS Sanitization
         |
         ▼
[ Step 4: Catalog Discovery & Fuzzy Search ]
  Keyword Search ("vntage") -> Typo-Tolerance Trigger -> Search Results Hydration
         |
         ▼
[ Step 5: Faceted Filtering & Sorting ]
  Filter: Size [M] + Price [$50-$150] + Brand -> Sort: "Newest Arrivals" -> URL Sync
         |
         ▼
[ Step 6: PDP Variant Matrix & Inventory Check ]
  Select Color Swatch -> Select Size -> Stock Status & Image Gallery Dynamic Swap
         |
         ▼
[ Step 7: Size Chart & Social Share Verification ]
  Open Size Guide Drawer -> Metric/Imperial Toggle -> Social Share Link Assertion
         |
         ▼
[ Step 8: Recommendation Profiling & Wishlist ]
  Heart/Wishlist Item -> Navigate Home -> Verify "Recommended For You" Personalization
```

2. Pre-conditions & Test Data Setup

Category	Prerequisite Details
Environment State	Staging environment with Elasticsearch / Algolia search index active, notification/email worker mock running.
Catalog State	Minimum 100 products loaded across diverse designers (e.g., "Atelier Noir", "Studio Minimal"), with multi-variant SKUs (Colors: Black, Cream, Emerald; Sizes: XS, S, M, L).
Inventory State	Specific test SKU: "Linen Oversized Blazer" where Variant `[Cream, M]` has stock = `0`, and Variant `[Black, M]` has stock = `5`.
Test User Persona	Non-registered email: `shopper.fashion.e2e_@stylehaven.io`, Password: `SecurePass#2026!`, Shipping Address: `452 Fashion Ave, Apt 12B, New York, NY 10018`.

3. Comprehensive Step-by-Step E2E Test Execution

Step #	Stage / Lifecycle Phase	Given / When / Then Test Steps	System Action & Internal Event	Expected Result & Verification Points	Pass/Fail Criteria
01	**User Registration & Validation**	**Given** user is on `/register` **When** user inputs first name, last name, unique email, and password `SecurePass#2026!` **And** clicks "Create Account" **Then** registration request is submitted	Client checks password policy (12+ chars, uppercase, lowercase, special char). API triggers `POST /api/v1/auth/register`, hashes password with bcrypt, generates verification token.	- Form validation passes with green indicators. - Verification screen appears: "Check your inbox". - Account record created in DB with `is_verified: false`.	**PASS:** Verification email dispatched with valid signed token within 5 seconds.
02	**Email Verification & Session Generation**	**Given** verification email is delivered **When** user clicks the activation link (`/verify?token=...`) **Then** the account is activated and user logs in	Token is validated by backend, setting `is_verified: true`. Backend issues `accessToken` (HTTP-only JWT) and `refreshToken`.	- UI displays "Email verified successfully!". - User redirected to `/account/overview`. - Global navbar updates from "Sign In" to user avatar with first name initial.	**PASS:** Authenticated session active; JWT stored securely in HTTP-only cookie.
03	**Profile & Default Address Configuration**	**Given** authenticated user is in `/account/profile` **When** user inputs phone `+1 212-555-0199` **And** adds shipping address: `452 Fashion Ave, Apt 12B`, New York, NY, `10018` **And** toggles "Set as Default Shipping Address" **Then** clicks "Save Changes"	Client executes `PUT /api/v1/user/profile` and `POST /api/v1/user/addresses`. Server validates postal code and strips malicious XSS tags.	- Notification toast: "Profile and address saved". - Address card displays "DEFAULT" badge. - Reloading `/account/profile` retains all saved values without data loss.	**PASS:** Address persisted; XSS injection payload sanitized; default flag set to true.
04	**Global Navigation & Search Engine (Typo Tolerance)**	**Given** user is on any storefront page **When** user enters typo keyword `vntage jaket` in the search bar and presses Enter **Then** search query executes	Request `GET /api/v1/catalog/search?q=vntage+jaket` sent to Elasticsearch/Algolia service. Fuzzy logic matches levenshtein distance <= 2.	- Search header displays: *"Showing results for **vintage jacket** (Did you mean: vintage jacket?)"*. - Results grid populates relevant outerwear items. - Zero-result empty state is NOT triggered.	**PASS:** Accurate results returned despite spelling mistakes; query time < 350ms.

Step #	Stage / Lifecycle Phase	Given / When / Then Test Steps	System Action & Internal Event	Expected Result & Verification Points	Pass/Fail Criteria
05	**Faceted Filtering & Sorting Matrix**	**Given** user is viewing search/catalog results **When** user selects filters: - **Category:** "Women's Blazers & Jackets" - **Designer / Brand:** "Atelier Noir" - **Size:** "M" - **Price:** "$50 - $150" **And** selects sort option: "Newest First" **Then** catalog query refreshes	URL parameters dynamically update to `/catalog?category=blazers&brand=atelier-noir&size=M&price=50-150&sort=newest` without full page refresh.	- Product grid dynamically filters to matching items only. - Filter badge chips display selected attributes with "x" remove options. - Result count counter reflects the filtered subset.	**PASS:** URL state matches applied filters; product grid displays only items satisfying all criteria.
06	**PDP Navigation & High-Resolution Asset Loading**	**Given** filtered product listing displays "Linen Oversized Blazer" **When** user clicks the product card **Then** browser transitions to Product Detail Page (`/products/linen-oversized-blazer`)	Next.js / frontend router navigates to PDP; `GET /api/v1/products/{id}` fetches full SKU matrix, breadcrumbs, size guide, reviews.	- Breadcrumb renders: `Home > Women > Blazers > Linen Oversized Blazer` . - Main image loads in high resolution with smooth placeholder blur-up. - Designer card displays "Sold by: Atelier Noir" with designer bio link.	**PASS:** Page loads within 1.5s; all images and descriptions match catalog DB.
07	**Variant Matrix & Out-of-Stock Handling**	**Given** user is on PDP for "Linen Oversized Blazer" **When** user clicks color swatch "Cream" **And** selects size button "M" (Configured out of stock) **Then** stock availability engine runs	Component checks variant matrix `SKU-BLAZER-CRM-M` where `stock_count == 0` .	- "Add to Cart" button immediately changes to disabled **"Out of Stock"**. - Subtext displays "Notify me when available" input box. - Image gallery swaps to photos of the **Cream** color variant.	**PASS:** System prevents adding zero-inventory variant; UI correctly reflects out-of-stock state.
08	**Variant Matrix & In-Stock Selection**	**Given** user is still on PDP **When** user clicks color swatch "Black" **And** clicks size button "M" (Configured in stock = 5) **Then** stock availability engine re-evaluates	Component checks `SKU-BLAZER-BLK-M` where `stock_count == 5` .	- "Add to Cart" button re-enables to active state. - Stock badge displays: *"Low Stock: Only 5 left!"*. - Image carousel switches to high-res images of the **Black** blazer.	**PASS:** Button enabled; stock counter matches inventory DB; correct variant SKU selected.

Step #	Stage / Lifecycle Phase	Given / When / Then Test Steps	System Action & Internal Event	Expected Result & Verification Points	Pass/Fail Criteria
09	**Interactive Size Guide & Social Sharing**	**Given** user wants to verify measurements **When** user clicks "Size Guide" link **And** toggles between "Inches" and "Centimeters" **And** closes drawer, then clicks "Share" icon (Twitter/Copy Link) **Then** interactions respond smoothly	Size drawer opens with accessible modal focus trap. Unit conversion recalculates table values dynamically. Share trigger copies clean canonical URL to clipboard.	- Unit conversion switches values accurately (e.g., Chest: 38" -> 96.5 cm). - Modal closes cleanly without page scroll displacement. - Clipboard receives valid URL: `https://stylehaven.io/products/linen-oversized-blazer`.	**PASS:** Size conversions mathematically correct; share link copied without tracking artifacts.
10	**Wishlist Persistence & Personalized Recommendations**	**Given** user is authenticated **When** user clicks the Heart icon on the PDP **And** navigates back to Home page (`/`) **Then** user checks "My Wishlist" and the "Recommended For You" feed	API sends `POST /api/v1/wishlist` with product ID. Recommendation engine logs user interest vector `[category: blazers, style: minimalist, designer: Atelier Noir]`.	- Heart icon fills with active color. - Item appears in `/account/wishlist`. - Home page carousel **"Recommended For You"** features complementary items (e.g., matching tailored trousers, linen skirts from Atelier Noir).	**PASS:** Wishlist item persisted across page navigation; recommendations adapt to browsing behavior.

4. Edge Cases & Boundary Validations within this Flow

1. **Session Resiliency Across Tabs:**

When User registers and logs in on Tab 1, opening Tab 2 should immediately reflect the authenticated state and cart/wishlist counters without requiring re-login.

2. **Concurrent Variant Depletion:**

If stock for Variant `[Black, M]` drops from 1 to 0 in another concurrent session while the user is looking at the PDP, attempting to click "Add to Cart" must query real-time stock and display a gentle warning: *"Sorry, this item was just claimed by another shopper"*.

3. **Deep Link / URL Direct Navigation:**

Copying the filtered catalog URL `https://stylehaven.io/catalog?brand=atelier-noir&size=M` into an incognito window must restore the exact same filters, sort order, and product grid.

4. **Special Character & Unicode Handling:**

Searching for designer names with accents or non-Latin glyphs (e.g., `Chloé`, `Renée`, `Ателье`) must resolve to the correct designer store without encoding corruptions (`%20`, `&`).

5. Playwright Automation Implementation Spec

```
// tests/e2e/user-account-catalog-flow.spec.ts
import { test, expect } from '@playwright/test';

test.describe('E2E Flow: User Account Onboarding to Product Catalog Discovery', () => {
  const timestamp = Date.now();
  const testEmail = `shopper_${timestamp}@stylehaven.io`;
  const password = 'SecurePass#2026!';

  test('Complete Shopper Journey: Register, Setup Profile, Search, Filter, Variant & Wishlist', async ({ page }) => {
    // 1. Onboarding & Registration
    await page.goto('/register');
    await page.fill('input[name="firstName"]', 'Sophia');
    await page.fill('input[name="lastName"]', 'Vance');
    await page.fill('input[name="email"]', testEmail);
    await page.fill('input[name="password"]', password);
    await page.click('button[type="submit"]');
    await expect(page.locator('.verification-notice')).toBeVisible();

    // 2. Simulate Token Verification & Login
    await page.goto(`/verify?token=mock_valid_token_${timestamp}`);
    await expect(page.locator('.navbar-user-avatar')).toContainText('S');

    // 3. Profile & Address Setup
    await page.goto('/account/profile');
    await page.fill('input[name="phone"]', '+12125550199');
    await page.fill('input[name="addressLine1"]', '452 Fashion Ave, Apt 12B');
    await page.fill('input[name="city"]', 'New York');
    await page.selectOption('select[name="state"]', 'NY');
    await page.fill('input[name="zip"]', '10018');
    await page.click('button:has-text("Save Changes")');
    await expect(page.locator('.toast-success')).toBeVisible();

    // 4. Global Fuzzy Search
    await page.fill('input[type="search"]', 'vntage jaket');
    await page.keyboard.press('Enter');
    await expect(page.locator('.search-correction')).toContainText('vintage jacket');

    // 5. Faceted Filtering
    await page.click('label:has-text("Atelier Noir")');
    await page.click('button[data-size="M"]');
    await page.selectOption('select[name="sort"]', 'newest');
    await expect(page).toHaveURL(/.*brand=atelier-noir.*size=M/);

    // 6. PDP Variant Validation (Out-of-Stock vs In-Stock)
    await page.click('.product-card:has-text("Linen Oversized Blazer")');
```

```
    await page.click('button[data-color="Cream"]');
    await page.click('button[data-size="M"]');
    await expect(page.locator('button.btn-add-to-cart')).toBeDisabled();
    await expect(page.locator('.stock-status')).toContainText('Out of Stock');

    // Switch to in-stock variant
    await page.click('button[data-color="Black"]');
    await expect(page.locator('button.btn-add-to-cart')).toBeEnabled();
    await expect(page.locator('.stock-status')).toContainText('Low Stock: Only 5 left!');

    // 7. Size Chart Drawer
    await page.click('button:has-text("Size Guide")');
    await expect(page.locator('.size-chart-drawer')).toBeVisible();
    await page.click('button:has-text("Centimeters")');
    await expect(page.locator('.metric-measurement')).toBeVisible();
    await page.click('.size-chart-drawer .btn-close');

    // 8. Wishlist & Recommendation Profiling
    await page.click('button.btn-wishlist');
    await expect(page.locator('button.btn-wishlist svg')).toHaveClass(/active|filled/);
    await page.goto('/');
    await expect(page.locator('#recommended-for-you .product-card')).toHaveCount(4);
  });
});
```